Kaushalya
The Skill University

Course: Bachelor of Computer Applications (B.C.A)

Real-Time Object Detection Project Report

Submitted By: kavan patel

Guided By: Ms. Shruti Mittal Vaidya

Date of Submission: November 2026

Dr. Parag Shukla

Head of the Department

2. Abstract

Real-Time Object Detection is a computer vision system that identifies and locates objects in live video streams. This project uses deep learning algorithms to detect objects such as people, cars, and other items in real time using a webcam. The system processes video frames and displays bounding boxes along with object labels and confidence scores. This technology is widely used in surveillance, autonomous vehicles, and smart systems.

📌 3. Introduction

Object detection is a technique in computer vision that allows machines to identify and locate objects in images or videos. Real-time detection means the system processes frames instantly using a camera.

This project uses:

- Python programming
- OpenCV for image processing
- YOLO (You Only Look Once) algorithm for fast detection

🎯 4. Objectives

- To detect objects in real-time using webcam
- To understand deep learning-based object detection
- To implement YOLO model
- To display bounding boxes and confidence score
- To improve accuracy and speed

🧠 5. Technology Used

- **Programming Language:** Python
- **Libraries:**
  - OpenCV
  - NumPy
  - Ultralytics YOLO
- **Model:** YOLOv8
- **Hardware:** Webcam / Camera

⚙ 6. Methodology / Working

1. Capture video from webcam
2. Read each frame
3. Pass frame to YOLO model
4. Model detects objects
5. Draw bounding boxes
6. Display object name + confidence
7. Show output on screen

🧮 7. Algorithm Used (YOLO)

YOLO (You Only Look Once) is a deep learning algorithm used for object detection.

Steps:

1. Divide image into grid
2. Predict bounding boxes
3. Assign class probabilities
4. Apply confidence score
5. Use Non-Max Suppression to remove duplicates

Formula:

Confidence Score = Probability × IOU

🔄 8. System Flow (Flowchart Explanation)

- Start
- Initialize camera
- Load YOLO model
- Capture frame
- Detect objects
- Display result
- Repeat until exit

8.code

```
from ultralytics import YOLO
import cv2
import random

# Load model
model = YOLO("yolov8s.pt")

# Start webcam (0 = default camera)
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("❌ Webcam not working")
    exit()

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Run detection
    results = model(frame)

    object_counts = {}

    for r in results:
        for box in r.boxes:
            conf = float(box.conf[0])
            if conf < 0.5:
                continue

            cls_id = int(box.cls[0])
            label = r.names[cls_id]

            # Count objects
            object_counts[label] = object_counts.get(label, 0) + 1

            x1, y1, x2, y2 = map(int, box.xyxy[0])

            # Random color
```

```
            color = [random.randint(0,255) for _ in range(3)]

            # Draw box
            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

            # Label
            text = f"{label} {conf:.2f}"
            cv2.putText(frame, text, (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                        color, 2)

    # Show counts on screen
    y_offset = 30
    for obj, count in object_counts.items():
        cv2.putText(frame, f"{obj}: {count}", (10, y_offset),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7,
                    (0, 255, 0), 2)
        y_offset += 30

    # Show frame
    cv2.imshow("🔥 Live Object Detection (YOLOv8)", frame)

    # Press 'q' to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Release resources
cap.release()
cv2.destroyAllWindows()
```

📊 11. Output

- Live video with detected objects
- Labels (person, car, etc.)
- Confidence score (e.g., 0.73 = 73% accuracy)

12. Advantages

- Fast detection
- High accuracy
- Works in real-time
- Useful in many industries

13. Limitations

- Needs good hardware
- Accuracy depends on lighting
- May struggle with small objects

🚀 14. Applications

- Security systems
- Self-driving cars
- Face detection
- Smart surveillance
- Traffic monitoring

🔮 15. Future Scope

- Add face recognition
- Add attendance system
- Improve accuracy with custom dataset
- Use GPU for faster processing

📃 16. Conclusion

This project successfully demonstrates real-time object detection using YOLO. It shows how deep learning can be applied in real-world applications to detect and classify objects efficiently.

17. References

- YOLO Documentation
- OpenCV Documentation
- Python Official Docs